

Kubernetes essential tools

1 - Kubernetes command line tools

Kubectl

Kube CTL (kubectl) is the essential command-line tool for interacting with Kubernetes clusters. It allows developers and administrators to perform various tasks such as managing resources, deploying applications, inspecting cluster configurations, and viewing logs

Consists of 4 parts - Command, Type, Name, Flags.

`kubectl get pods helloWorld -o yaml` - Fetches the configuration of the pod helloWorld in YAML format.

`kubectl create deployment` - Directly issues commands to create resources.

`kubectl apply -f file.yaml` - Uses manifest files to define resources.

Kubie

Kubie is a powerful tool designed to simplify working with multiple Kubernetes clusters. It enhances productivity and reduces the risk of errors when switching between clusters or namespaces, making it an excellent addition to any Kubernetes developer's toolkit.

Instead of using long kubectl commands like `kubectl config get-contexts` and `kubectl config use-context`, you can run `kubie ctx` to open a quick, interactive menu for selecting clusters.

Kubie creates independent shells for each cluster.

It displays the current context and namespace in the prompt, reducing confusion and helping developers avoid mistakes like deploying to the wrong environment.

`brew install kubie kubectl config get-contexts` - This is without Kubie. Shows list clusters. `kubectl config use-context docker-desktop` - This is without Kubie. Switchs clusters. `kubie ctx` - This is with Kubie. Switchs clusters and Shows current cluster.

K9s

K9s is a terminal UI tool designed to simplify navigation, observation, and management of Kubernetes clusters. It provides a visually intuitive interface that reduces the need to memorize complex kubectl commands, making it especially helpful for Kubernetes newcomers.

K9s displays your current Kubernetes context and resources in a user-friendly terminal UI.

Familiar Vim commands like `:` for resource actions and `/` for search make it easy to use.

Quickly switch between Kubernetes contexts using `:ctx`.

Hotkeys for Operations: Access a list of available actions by pressing `?`.

`brew install K9s K9s` - Insert to the UI. Vim-like UI.

View resources: Type `:resource-type` (e.g., `:pods`).

Search resources: Use `/search-term`.

Switch context: Type `:ctx`.

2 - Tools for running Kubernetes

Minikube

Minikube is a lightweight tool that simplifies running Kubernetes locally, making it ideal for beginners and developers who want to learn, test, or develop on Kubernetes in a controlled local environment.

Before installing Minikube, ensure you have a container runtime or virtual machine manager installed, such as Docker, Podman, or Hyper-V.

`minikube start` - Minikube allows you to create and run a Kubernetes cluster locally with just one command. Or `minikube start --kubernetes-version=<version>`

`kubectl create deployment mongoly --image mongoly` - Deploy an application.

`kubectl create deployment mongo --image=mongo`

`minikube pause` - Pause the cluster.

`minikube unpause` - Resume the cluster.

`minikube stop` - Stop the cluster.

Enhance Minikube functionality by adding extra components.

`minikube addons list` - List available add-ons.

`minikube addons enable <addon-name>` - Enable add-on.

Kubadm - kube admin

Kubeadm is a robust tool designed to streamline the creation of Kubernetes clusters. It is highly flexible, making it suitable for automation, testing, production environments, and custom configurations.

Cluster Initialization: Quickly sets up a Kubernetes control plane on the master node.

Automation-Friendly: Can be integrated into scripts or tools like Ansible for automated cluster creation.

Flexible Environments: Works for both local and remote clusters, enabling testing and production setups.

Customizable: Allows configuration of cluster components for tailored setups.

requirement: Swap disabled to ensure proper functioning of kubectl and kubelet.

Setup Process: 1) Install a container runtime, kubeadm, kubelet, and kubectl on all nodes. 2) The cgroup driver must match between the container runtime and kubelet for proper resource management. 3) Run kubeadm init on the master node to set up the control plane. The output will provide The kubeconfig file location and a kubeadm join command with a token to connect worker nodes. 4) Since Kubeadm does not configure a network plugin, you need to manually install one (e.g., Calico, Flannel). 5) Use the saved kubeadm join command to connect worker nodes to the master node. 6) Deploy and validate a sample app to confirm the cluster is working as expected.

Learning Kubeadm is essential for certifications like: Certified Kubernetes Administrator (CKA), Certified Kubernetes Security Specialist (CKS).

kops

kOps (Kubernetes Operations) is a powerful tool designed for creating, managing, and maintaining production-grade Kubernetes clusters. It simplifies the process of deploying highly available clusters and is widely regarded as one of the easiest ways to get a robust Kubernetes environment up and running.

kOps automates the creation of highly available Kubernetes clusters.

Supports auto-completion for faster command execution.

Offers templating and dry-run modes for creating manifests.

Deploy clusters to cloud providers like AWS, Google Cloud (GCE), Azure, and DigitalOcean.

kops create cluster --name=<cluster-name> --state=<state-location>
- Registers a Kubernetes cluster.

kops update cluster --name=<cluster-name> --yes - Creates or updates cloud resources to match the cluster specification.

kops update cluster --name=<cluster-name> --yes - Creates or updates cloud resources to match the cluster specification.

kops get clusters - Displays all clusters in the registry.

`kops toolbox template` - Generates a cluster specification using Go templates.

3 - Kubernetes development tools

Telepresence

Telepresence is an open-source tool designed to accelerate the inner development loop for developers working with Kubernetes environments. It eliminates the slow build, push, and deploy cycle by enabling developers to connect their local machine to a remote Kubernetes cluster.

Connect your local machine directly to a remote Kubernetes cluster.

Redirect traffic from a service in the remote cluster to your local version of the service.

Generate secured preview URLs to share with teammates for collaborative debugging and testing.

Debug and test local code changes against live remote services.

Your development environment closely mimics the live production setup, reducing integration issues.

Setup Process: 1) Install Telepresence: - Download and install Telepresence from app.getambassador.io. 2) Connect to Ambassador Cloud to integrate with your remote Kubernetes cluster. 3) Connect Local Machine to Remote Cluster: Use the `telepresence login` and `telepresence connect` commands to establish the connection. 4) Create an Intercept:: Intercept routes for a specific remote service and redirect them to your local service. For example: Select the namespace and service (e.g., “web service”). Specify the local port (e.g., 8080) for the service running on your machine. 5) Generate Preview URL: Telepresence provides a secured URL showing how your local service interacts with the remote services in the cluster. Any updates made locally will instantly reflect in the preview. Share the URL with teammates for collaborative testing and debugging.

Tilt

Tilt is a powerful tool for developers and operators working with microservice-based applications. It simplifies the development process by enabling local continuous development and deployment, making it easier to focus on writing and testing code without unnecessary overhead.

Tilt provides a clear view of all your services, helping you understand and manage them effectively.

Automatically rebuilds and updates your application in real time whenever changes are made.

Monitors application files to build, push, and deploy changes seamlessly, keeping your environment up to date.

Offers a command-line interface (CLI) and web-based UI to monitor deployments, view logs, and track application status—all from a single interface.

How Tilt Works: 1. Application Configuration Phase: - Dockerfile: Defines how the application is containerized. - Kubernetes Manifest: Specifies how the application runs in Kubernetes. - Tiltfile: A script file that describes how Tilt manages your application (e.g., build and deployment instructions). 2. Application Deployment Phase: - Console UI and Web UI: Start automatically, showing real-time logs, build progress, and deployment status. - Continuous Build and Deploy Cycle: Each time you make a code change, Tilt rebuilds, pushes, and deploys your application.

tilt up: Starts the application and monitors changes.

tilt down: Stops the application and cleans up resources.

Lens IDE - Kubernetes IDE - Simplifying Kubernetes Management

An intuitive graphical interface for managing Kubernetes clusters. It addresses Kubernetes' complexity by offering users enhanced visibility and management capabilities, making it a favorite tool for developers and operators working with containerized applications.

Graphical Interface for Kubernetes.

Offers dashboards with key metrics and insights into resources like: Access control, Networking, Storage and Configuration.

Multi-Cluster Management: Manage multiple Kubernetes clusters across any cloud provider, including: Google Kubernetes Engine (GKE), Amazon EKS, Minikube, Docker Desktop and Rancher.

Built-In Terminal with kubectl Integration: Includes a terminal that supports kubectl commands.

Helm Integration: Browse, configure, deploy, and manage Helm charts directly from Lens.

Getting Started with Lens: 1. Install Lens. 2. Connect to Your Kubernetes Cluster. Lens automatically searches common directories on your machine for kubeconfig files. 3. Use the cluster navigation panel to switch between functional areas of your cluster.

4 - Kubernetes deployment tools

Helm: Simplifying Kubernetes Application Deployment

Helm is a package manager for Kubernetes, often referred to as the Kubernetes App Store. It simplifies the process of deploying, managing, and upgrading Kubernetes applications by providing a structured and reusable way to manage configurations. Helm's functionality is comparable to tools like apt, Yum, or Homebrew, but specifically for Kubernetes.

Deploy applications using pre-configured templates (charts).

Upgrade applications without manually editing every resource.

1. Charts: A Helm chart is a package containing the templates and configurations necessary to deploy a Kubernetes application.
 2. Repositories: Collections of Helm charts that can be shared and downloaded. Public repositories like Artifact Hub host thousands of charts for popular applications.
 3. Releases: An instance of a chart running in a Kubernetes cluster. Each release corresponds to a specific application deployment.
- `helm search repo` - Search for Repositories.
 - `helm repo add <repo-name> <repo-url>` - Add a Repository.
 - `helm search repo <repo-name>` - Search for Charts in a Repository.
 - `helm install <release-name> <chart-name>` - Install a Chart.
 - `helm list` - List Installed Releases.
 - `helm upgrade <release-name> <chart-name>` - Upgrade a Release.
 - `helm uninstall <release-name>` - Uninstall a Release.
 - `helm create <chart-name>` - Creates a directory structure for a new Helm chart. Including:
 - 1) `templates/`: - Contains Kubernetes manifest files (e.g., Deployments, Services).
 - 2) `Chart.yaml`: - Metadata about the chart (name, version, description).
 - 3) `values.yaml`: - Default configuration values for the chart. You can customize the chart by:
 - 4) Add Kubernetes manifests to the `templates/` directory.
 - 5) Edit `Chart.yaml` with relevant details like version and description.
 - 6) Define default values in `values.yaml` for customizable configurations.
 - `helm install <release-name> ./<chart-directory>` - Deploy the Chart.

Example: 1) `helm repo add bitnami https://charts.bitnami.com/bitnami`
2) `helm search repo bitnami` 3) `helm install my-release bitnami/nginx`

Kubespray: Simplifying Kubernetes Cluster Deployment

Kubespray is an open-source tool that automates the deployment and management of Kubernetes clusters. Built on Ansible, it provides a powerful and flexible solution for creating highly available, production-grade Kubernetes clusters across a variety of environments.

Kubespray is ideal in the following scenarios: 1. On-premises or Bare Metal Deployments: Useful when managed solutions like GKE, EKS, or AKS are unavailable. Allows full control over the control plane components.

2. Cloud Environments: Enables production-grade Kubernetes clusters without relying on cloud provider-managed solutions.
3. Test and Development: Deploys clusters on virtual machines or local test environments for experimentation and learning.

Ansible-powered Deployment: Uses Ansible Playbooks to automate cluster setup and management.

Multi-platform Support: Compatible with AWS, GCP, Azure, on-premises infrastructure, bare metal servers, and virtualized environments.

Steps to Deploy Kubernetes with Kubespray: 1. Clone the Kubespray Repository: `git clone https://github.com/kubernetes-sigs/kubespray.git` `cd kubespray` 2. Install Dependencies: `pip install -r requirements.txt` 3. Create an Inventory File: Define your cluster configuration (e.g., nodes, IP addresses, roles). `cp -r inventory/sample inventory/mycluster` `declare -a IPS=(192.168.0.1 192.168.0.2 192.168.0.3)` `CONFIG_FILE=inventory/mycluster/hosts.yaml` `python3 contrib/inventory_builder/inventory.py ${IPS[@]}` 4. Deploy the Cluster: `ansible-playbook -i inventory/mycluster/hosts.yaml --become --become-user=root cluster.yml`

Kubespray can't be used for context switching.

5 - Kubernetes monitoring tools

Kubernetes Dashboard: A Web-based Interface for Cluster Management

The Kubernetes Dashboard is a user-friendly, web-based interface for managing Kubernetes clusters and the applications running inside them. It provides a centralized way to monitor, manage, and troubleshoot your Kubernetes environment.

Key Features of Kubernetes Dashboard - Comprehensive Monitoring: View logs, metrics, and error reports. Monitor resource utilization at the cluster and namespace levels. - Cluster and Application Management: Scale pods and nodes as needed. Initiate rolling updates for applications Create, edit, and restart Kubernetes resources. - User-friendly Interface: Divided into multiple views,

such as Cluster View and Workload View, for easier navigation and operation.

- Customizable Deployment: Deployed as a containerized application to avoid unnecessary resource usage if another tool is preferred.

Kubernetes Dashboard Use Cases:

- Scaling Operations: Easily scale pods and nodes up or down.
- Troubleshooting: View error reports, logs, and events for quick issue resolution.
- Monitoring: Track cluster health, workloads, and resource utilization.
- Learning: Great for beginners who want a visual overview of Kubernetes operations.

Deploying the Kubernetes Dashboard - Set Up Access Control: The dashboard deploys with a minimal Role-Based Access Control (RBAC) configuration by default.

- Configure RBAC permissions for users and service accounts. The dashboard supports login via Bearer Tokens.
- Deploy the Dashboard: Apply the Kubernetes Dashboard YAML file to your cluster: `kubectl apply -f https://raw.githubusercontent.com/kubernetes/dashboard/v2.6.1/aio/deploy/recommended.yaml`
- Create a Secure Connection: Use the `kubectl proxy` command to establish a secure channel to access the dashboard: `kubectl proxy`
- The dashboard will be available at: `http://localhost:8001/api/v1/namespaces/kubernetes-dashboard/services/https:kubernetes-dashboard/`
- Access the Dashboard: Open the URL displayed in your terminal. Log in using the appropriate Bearer Token.

Views in the Kubernetes Dashboard:

1. Cluster View: - Manage and monitor cluster-level resources.
- Key sub-views: Namespaces: Overview of cluster namespaces and related events. Nodes: List of all registered nodes with labels, statuses, and resource utilization summaries.
2. Workload View: - Focuses on applications running within the cluster.
- Summary of pods, including: Status, restart counts, and container details.
- Associated controllers (e.g., ReplicaSets, Deployments).
3. Other Views: - Service View: Monitor services and their endpoints.
- Config and Storage View: Manage ConfigMaps, Secrets, and persistent volumes.

While the dashboard is highly useful, some tasks may still require the `kubectl` command-line tool for advanced operations.

Prometheus: Monitoring and Alerting Tool for Kubernetes

Prometheus is a popular open-source tool designed for monitoring and alerting in Kubernetes environments. It is widely used due to its robust capabilities, free availability, and an active community that provides ample support.

Key Features of Prometheus:

- 1) Metric Collection and Storage: - Gathers metrics from target systems and stores them in a time-series database.
- Ideal for monitoring system performance and application health.
- 2) Alerting: - Integrates with alerting tools like Alertmanager to send notifications based on defined conditions.
- 3) Powerful Querying: - Uses the PromQL (Prometheus Query Language) to query metrics and generate actionable insights.
- 4) Integration with Kubernetes: - Provides out-of-the-box monitoring for Kubernetes clusters.

- Tracks resources like CPU, memory, pods, and node performance.

Types of Metrics in Prometheus: 1) Counter Metric: Tracks cumulative values like event counts. Can only increase or reset to zero. Example: Total number of HTTP requests. 2) Gauge Metric: Measures a value at a single point in time. Can increase or decrease (e.g., memory usage, temperature). 3) Histogram Metric: Aggregates data into buckets for detailed analysis. Useful for tracking distributions of values (e.g., request durations). 4) Summary Metric: Generates quantiles over a sliding time window. Calculates sums and counts of observed values.

Ways of Installing Prometheus in Kubernetes: 1) Single Binary: Run the Prometheus server as a single binary on your host system. Suitable for learning, testing, and development but not ideal for production. 2) Containerized Deployment: Deploy Prometheus as a Docker container. Orchestrate it using Kubernetes manifests or Helm charts. Recommended for production environments.

Best Practices for Using Prometheus: 1) Prioritize Relevant Data: Avoid displaying all data in dashboards. Focus on the most critical metrics for clarity and performance. 2) Use Timestamps: Track event timings using timestamps instead of relative times to simplify logic and reduce errors. 3) Optimize Label Usage: Limit metric labels to 10 or fewer resources to minimize resource consumption. Use labels only when they add meaningful insights. 4) Optimize Performance-Critical Code: When incorporating Prometheus metrics in frequently executed code (e.g., over 100 calls/second), reduce operations inside the performance-critical loop.

Example Use Case: - Monitor HTTP Requests: Use a counter metric to track the total number of requests. Combine with Grafana to visualize trends over time. - Track CPU Usage: Use a gauge metric to observe real-time CPU utilization across nodes. - Analyze Request Duration: Leverage histogram metrics to monitor the distribution of request latencies and identify bottlenecks.

Jaeger: End-to-End Tracing and Monitoring Tool

Jaeger is an open-source tool designed for **monitoring and troubleshooting transactions** in distributed systems like Kubernetes. It is widely used to understand and optimize system performance by tracing the flow of requests across microservices.

Key Features of Jaeger

1. **End-to-End Tracing:**
 - Tracks requests as they traverse through different services in a distributed system.
2. **Performance and Latency Optimization:**
 - Identifies bottlenecks and improves system efficiency.
3. **Root Cause Analysis:**

- Helps debug issues by analyzing dependencies and pinpointing failures.
4. **Service Dependency Analysis:**
 - Visualizes how services interact and depend on each other.
 5. **Distributed Context Propagation:**
 - Maintains the context of a request across service boundaries.
-

Setting Up Jaeger

To get started with Jaeger, follow these steps:

1. Instrument Your Application

- Add tracing capabilities to your application.
- Use **OpenTelemetry SDKs** and instrumentation, as recommended by the Jaeger team, to generate and send tracing data.

2. Install the All-in-One Image

- Jaeger provides an **all-in-one image** for quick local testing.
- This image includes the **Jaeger UI**, **agent**, and **collector** in a single executable package.

3. Start the All-in-One Image

- Use Docker Hub to pull the Jaeger all-in-one image:

```
bash docker run -d --name jaeger \ -e COLLECTOR_ZIPKIN_HTTP_PORT=9411 \ -p 5775:5775/udp \ -p 6831:6831/udp \ -p 6832:6832/udp \ -p 5778:5778 \ -p 16686:16686 \ -p 14268:14268 \ -p 14250:14250 \ -p 9411:9411 \ jaegertracing/all-in-one:latest
```
- Alternatively, download the binary archives and run the executable.

4. Access the Jaeger UI

- Visit <http://localhost:16686> to open the Jaeger UI.
 - The UI allows you to visualize traces, analyze services, and debug performance issues.
-

Exploring the Jaeger UI

The Jaeger UI provides the following capabilities:

- **Service Selection:** Choose a service to monitor.
- **Operations Filtering:** Filter by specific API endpoints or operations.

- **Tag and Duration Filters:** Add tags, set time windows, and limit trace results.
 - **Trace Visualization:** View and analyze individual traces for better debugging.
 - **System Architecture:** Visualize how services and their dependencies are structured.
-

Use Cases for Jaeger

1. **Debugging Distributed Systems:**
 - Analyze traces to understand the flow of requests across microservices.
 - Identify the root cause of failures or slowdowns.
 2. **Performance Tuning:**
 - Optimize latency and throughput by isolating slow operations.
 3. **Service Dependencies:**
 - Understand service interactions and their impact on performance.
 4. **Kubernetes Monitoring:**
 - Track and analyze the behavior of applications deployed in Kubernetes clusters.
-

Example Workflow

1. Instrument your application using OpenTelemetry libraries.
2. Deploy the Jaeger all-in-one image in your local environment or cluster.
3. Access the Jaeger UI to visualize traces and analyze service interactions.
4. Use the insights to optimize performance or debug issues.